

灵犀知径系统设计说明书

1、系统体系架构

灵犀知径平台采用“端云分离、多语言微服务协同”的现代分布式架构设计，旨在通过不同技术栈的有机融合，发挥各组件在特定领域的性能优势。如图 2.1 所示，系统在逻辑上划分为表现层、业务应用层、智能计算层与数据持久层四大核心板块，构建起从前端交互感知到后端深度推理的完整服务闭环。这种架构设计不仅确保了系统在大规模并发访问下的高可用性，更通过容器化沙盒机制实现了算法迭代与业务逻辑的深度解耦，为垂直领域 AI 应用的稳定性提供了坚实的工程支撑。

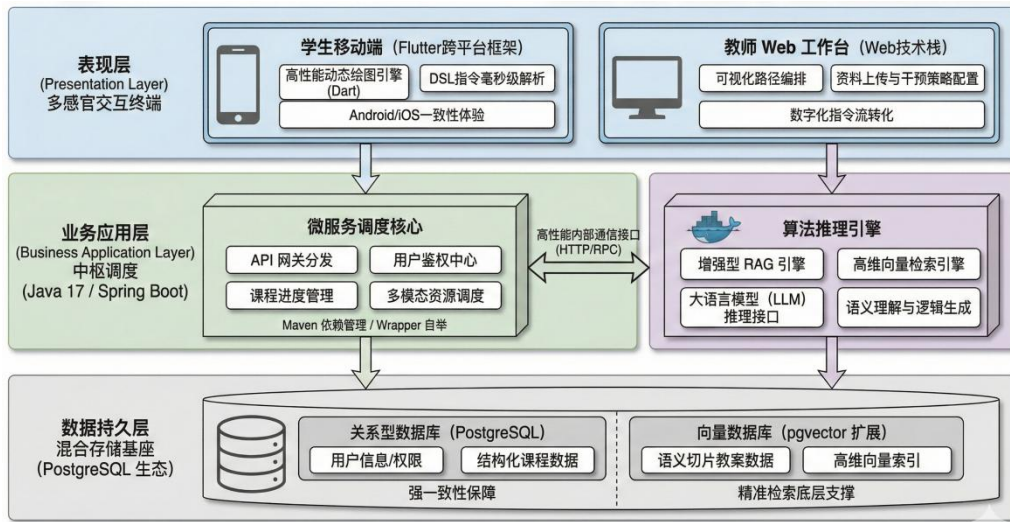


图 1.1 灵犀知径平台总体架构图

在表现层的构建中，系统针对学生端与教师端的不同诉求进行了差异化设计。移动端采用 Google Flutter 跨平台框架，基于 Dart 语言实现了一套高性能的动态绘图引擎，确保了 Android 与 iOS 端拥有完全一致的流畅体验与交互逻辑，为实现亚秒级的 DSL 指令解析与画布重绘提供了硬件加速支持。与此同时，教师端则依托 Web 技术栈构建可视化的路径编排工作台，方便教学专家高效地进行资料上传与干预策略配置，从而将复杂的教学经验转化为直观的数字化指令流。

系统的中枢神经由业务应用层与智能计算层共同驱动。业务应用层基于 Java

17 与 Spring Boot 框架构建，承担着用户鉴权、课程进度管理、多模态资源调度及 API 网关分发等核心任务，通过严格的 Maven 依赖管理与 Maven Wrapper 自举机制，保障了业务逻辑在生产环境中的稳健运行。与其深度协同的智能计算层则由独立的 Python 微服务集群构成，运行在完全隔离的 Docker 容器中。该层集成了增强型 RAG 引擎、高维向量检索引擎以及大语言模型推理接口，专注于处理复杂的语义理解与逻辑生成任务。两层之间通过高性能的内部通信接口进行数据交换，实现了业务稳定性与算法灵活性之间的有效平衡。

数据持久层作为支撑系统高并发访问的基石，采用了混合存储策略以应对多维数据需求。关系型数据库 PostgreSQL 负责存储用户信息、权限及结构化课程数据，确保了核心业务数据的强一致性；与此同时，通过集成 pgvector 扩展插件，该层级专门用于存储经过语义切片的教案数据与高维向量索引，为启发式助学过程中的精准检索提供底层算法支撑。这种将关系型映射与向量检索高度集成的存储模式，构建了一个具备语义感知能力的知识底座，不仅确保了教师原始教案的资产安全，也为灵犀知径的智能化服务提供了极高的数据可靠性与查询效能。

2、系统功能结构

系统的总体功能根据用户角色（学生/教师）和学习业务闭环，自顶向下划分为四大核心模块。以下为系统的总体功能层次结构图：

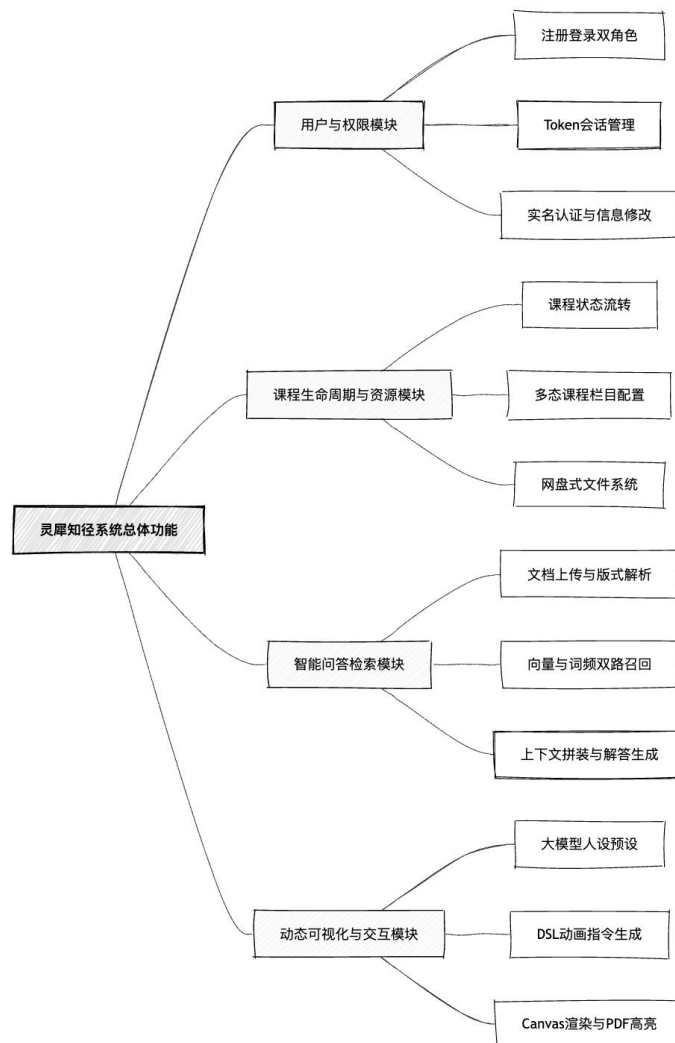


图 2.1 灵犀知径系统功能结构图

2.1 用户与权限模块

本模块是系统的安全门户，负责所有用户的身份管理、认证授权以及基础信息的维护。为了保障系统的安全性与数据的隔离性，我们设计了以下核心功能：

功能名称	详细说明
双角色账	系统原生支持“学生（STUDENT）”与“教师（TEACHER）”两种独立

户与加密认证体系	身份的注册与登录。支持通过手机号+验证码快速登录，以及传统的账号+密码形式登录。考虑到安全性，密码在前端传输时使用 RSA 公钥进行非对称加密，后端则采用 BCrypt 算法进行哈希加盐存储，从根源上杜绝数据库脱库导致的明文密码泄露风险。
基于 JWT 的无状态会话管理	采用 JSON Web Token 实现前后端分离架构下的单点登录与长效状态保持。为解决原生 JWT 无法主动强制下线的痛点，系统结合 Redis 实现了 Token 黑名单机制与双 Token（Access Token + Refresh Token）无感续签策略，既保证了账号安全性，又避免了学生在听课或答疑过程中频繁掉线。
细粒度的 RBAC 权限与接口拦截	系统后端配置了严格的全局权限拦截器体系。接口访问被划分为四个鉴权级别：公开（如登录注册）、需登录（如修改个人资料）、需特定角色（如“创建课程”接口仅限 TEACHER 角色调用），以及需课程权限（如访问特定课件或发起 AI 提问时，不仅要求已登录，还必须联合查询数据库确认该学生处于该课程的已加入状态），任何越权访问都会被网关直接拒绝。
学校与机构归属管理	在底层数据模型中，用户体系与“学校（School）”实体深度绑定。学生和教师在注册时需选择或绑定所在院校，这一设计不仅为首页“本校课程”的瀑布流推荐提供了精准的数据过滤支持，也为未来系统的多租户 SaaS 化和各校数据统计隔离奠定了基础。
教师专有实名认证	为了确保平台上教学资源的权威性与质量，教师账号在正式发布对外开放的课程前，可进入专门的流程进行实名认证登记（如上传教职工证明材料或进行身份信息核验）。未认证的教师账号可能受到建课数量或发布范围的限制。
个人中心与基础信息维护	提供完整的个人资料管理闭环，包括用户基础信息修改、头像上传（对接云端对象存储服务并返回 URL）、手机号换绑验证，以及安全的密码找回与重置功能。

表 2-1 用户与权限模块功能表

2.2 课程生命周期与资源模块

本模块旨在为教师提供一套完备的课程搭建与资料管理工具，为学生提供清晰的资源访问入口。作为教学活动的核心载体，该模块实现了以下关键能力：

功能名称	详细说明
全生命周期状态流转	涵盖课程从“预发布 (PRE_PUBLISH)”、“进行中 (IN_PROGRESS)”、 “已完结 (COMPLETED)”到“隐藏 (HIDDEN)”的完整生命周期 流转。系统底层结合不同的准入策略（自由加入/需审核加入），自 动动态计算并收紧或放开学生端在发现页的可见性与加入权限。 教师可自由灵活地配置课程内页的栏目体系与排序策略。系统支持 多种基础栏目类型：用于发布教学大纲或图文说明的“富文本公告 栏目配置 栏”、用于管理结构化课件资料的“网盘存储栏”，以及用于大模型智 能交互的专属“AI 答疑栏”。 在资料存储栏目中，实现了一套类似企业网盘的树形文件管理系 统。后端基于关系型数据库的自关联表结构（父子节点 ID 关联） 设计，并采用递归算法与懒加载机制支持多层级文件夹的快速展 开。支持文件的上传、重命名、移动与防误删等操作，方便教师结 构化地归档海量教学资源。 为了防止课件资源被恶意爬取或越权下载，所有的文件实体均对接 云端存储与 云端对象存储（如 OSS/MinIO）。后端在生成文件下载链接时采用 防盗链控制 临时预签名 URL（Pre-Signed URL）策略，确保只有具备该课程访 问权限的用户才能在一定时效内获取真正的资源内容。
云盘式多级文件系统	

表 2-2 课程生命周期与资源模块功能表

2.3 智能问答检索模块 (RAG)

这是本系统的智能化核心后台模块，负责将静态的讲义转化为 AI 可以理解的知识库，解决大模型“事实幻觉”的问题，并为前端提供精准的“图文溯源”

能力。

功能名称	详细说明
非结构化文档版式解析	教师上传 PDF 或 PPT 等课件后，系统后端调用强大的文档解析与版式分析工具（如 MinerU），自动且精准地剥离文档中的正文文本、标题、逻辑段落甚至数学公式，并提取记录每一个元素在原文档页面上的物理绝对坐标 (Bounding Box 边界框)，为后续的“原文高亮”打下坐标基准。 解析出的大段长文本会被系统的智能切片模块 (Chunking) 按照语义和段落长度进行合理切分。随后，调用高性能的 Embedding 模型将这些文本切片转化为高维语义向量，并统一持久化到 PostgreSQL 的 pgvector 扩展库中，供实时检索。
双路召回混合检索引擎	采用行业领先的混合召回策略以保障检索召回率。当学生提问时，系统会兵分两路：一路计算提问与课件知识库的“语义向量余弦相似度”（善于寻找意思相近但表述不同的段落）；另一路利用分词工具进行“BM25 词频全文检索”（善于精准匹配课件中的专业术语或关键符号）。
RRF 排序与黄金上下文拼装	系统使用倒数排名融合 (RRF, Reciprocal Rank Fusion) 算法将双路召回的结果进行交叉打分与重排，提取出最核心的 Top-N “黄金上下文”。最后，将这些课件原文、对应的坐标参数与学生的问题通过特定的 Prompt 模板拼接在一起发送给大模型，利用系统指令 (System Prompt) 强制要求 AI “只依据给定的课件材料进行解答，并务必在答案中附带引用坐标标签”。

表 2-3 智能问答检索模块功能表

2.4 动态可视化与交互模块

本模块承载了系统最具特色的前端交互体验，能够将抽象的文字逻辑变得动态且可视化，极大地降低了学生的认知门槛。

功能名称	详细说明
教师端专属 Prompt 配置	教师可以在后台专门为某门课程配置特定的 System Prompt（系统人设与级提示词）。例如，设定“请用引导启发的方式回答，不要直接给出最终代码”或“请扮演一位严厉但细心的大学教授”。这一机制能在底层灵活且无感地限定 AI 的解答风格与教学边界。
自研 DSL 动态指令流生成	针对算法演示或逻辑推导等复杂场景，系统要求大语言模型在输出文本解答的同时，按照我们预设的专有语法结构（DSL），生成一套包含坐标、颜色、形状与动作的结构化 JSON 指令流，作为驱动前端动画的数据载体。
Canvas 引擎与实时动画渲染	Flutter 移动端在接收到后端返回的 DSL 指令流后，内置的自研 Canvas 渲染引擎会对其进行实时解析，并在界面下方的动态画布上绘制出连贯的动画过程（如数组元素的冒泡交换、二叉树节点的动态链接等），实现图文并茂的直观教学。
智能 PDF 联动与图文精准溯源	在 AI 回复的内容中，如果包含了检索模块返回的原始坐标参数，前端系统会自动驱动底层的 PDF 视图引擎跳转到对应的页码，并在原课件的对应物理位置（如某行公式或某段定义）上方叠加半透明的高亮遮罩，帮助学生快速回到书本，实现知识的闭环溯源。

表 2-4 动态可视化与交互模块功能表

3、系统用例的时序图

3.1 时序图设计

本系统最核心的业务流程是：学生针对某门课程提出疑问，系统结合教师上传的课件进行检索，最终返回解答，并在移动端高亮课件对应的原图位置。

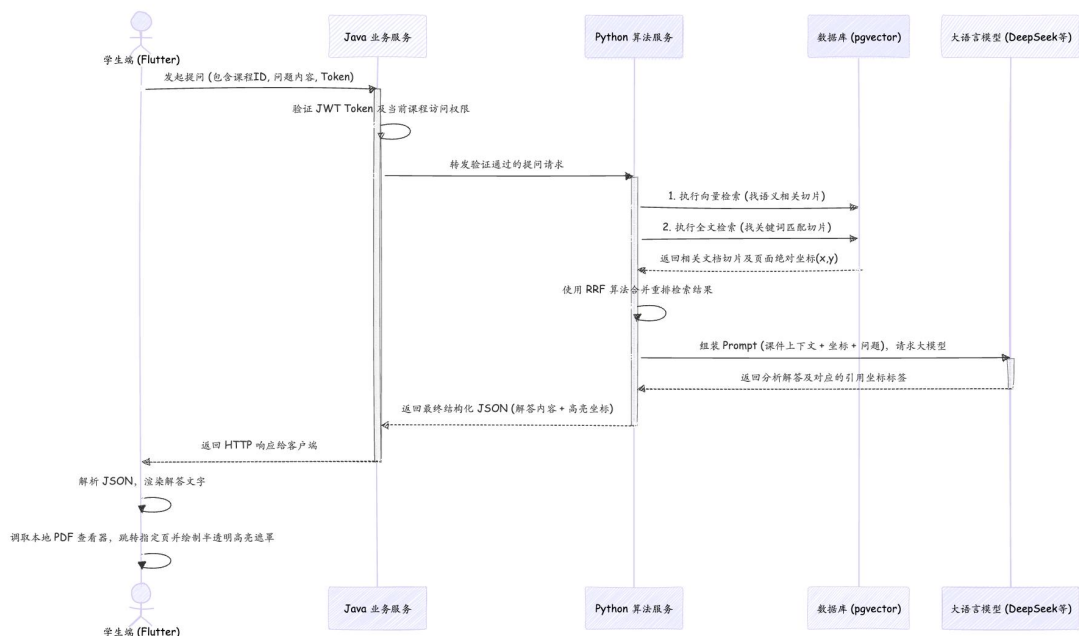


图 3.1 系统用例时序图

3.2 时序图详细说明

在鉴权与转发阶段, 学生在移动端点击发送问题, 请求先到达 Java 后端。Java 后端作为网关和权限中枢, 解析 JWT Token, 确保该学生确实加入了该课程。

验证通过后, 进入双路召回检索阶段。Python 端接手请求后, 兵分两路查询数据库: 一路通过把问题转为向量, 用余弦相似度找意思相近的段落; 另一路通过中文分词, 找包含相同专业术语的段落。

随后进行上下文拼接与生成。Python 端把两路找出来的“最相关课件内容”及其“在课件上的原始坐标”提取出来, 跟学生的问题拼在一起, 告诉大模型: “请根据以下老师的课件内容回答学生问题, 并在答案中附带你引用了哪条坐标”。

最后是前端响应与溯源渲染环节。大模型生成完毕后, 结果沿原路返回移动端。Flutter 移动端拿到结果后, 将文本显示在聊天气泡中; 同时根据返回的 (x, y) 相对坐标参数, 在下方的 PDF 控件中自动翻到对应页, 并计算物理像素, 用一个黄色图层将原课件上的公式或文字高亮框出。

4、复杂功能的算法设计

4.1 RAG 双路召回与重排算法设计

解决通用大模型容易“胡说八道”的问题，必须强制它根据老师发的课件回答，并且还要算出这段话在课件的哪里，方便学生对照复习。对此，我们设计了双路召回和重排算法：

算法伪代码表示如下：

```
// 算法入口：获取提问的最终“黄金上下文”
FUNCTION GetGoldenContext(user_question, course_id):
// 步骤 1：将用户提问转换为高维语义向量（例如 1024 维）
query_vector = EmbeddingModel.encode(user_question)
// 步骤 2：执行语义向量检索（基于 pgvector 余弦相似度）
// 找出该课程下，意思最相近的 Top 10 切片
vector_candidates = Database.execute("
SELECT id, text_content, page_num, bbox_coordinates
FROM course_files
WHERE course_id = @course_id
ORDER BY text_embedding <=> @query_vector LIMIT 10
")
// 步骤 3：执行关键词精准检索（基于 pg_jieba 和 FTS）
// 找出该课程下，包含相同专业术语的 Top 10 切片
keyword_candidates = Database.execute("
SELECT id, text_content, page_num, bbox_coordinates
FROM course_files
WHERE course_id = @course_id
AND text_content MATCH @user_question LIMIT 10
")
// 步骤 4：使用 RRF（Reciprocal Rank Fusion 倒数排名融合）算法合并结果
combined_scores = Dictionary<id, float>()
// 惩罚常数 k 一般取 60
k = 60
FOR index, candidate IN vector_candidates:
combined_scores[candidate.id] += 1.0 / (k + index + 1)
FOR index, candidate IN keyword_candidates:
```

```
combined_scores[candidate.id] += 1.0 / (k + index + 1)
// 步骤 5: 根据总分倒序排序, 取最终的前 3 个切片
final_top_3 = SortByScoreDescending(combined_scores).Take(3)
RETURN final_top_3
END FUNCTION
```

之所以要设计双路召回, 是因为单一方法有缺陷。比如学生搜“二叉树”, 如果只用向量检索, 可能会搜出“图结构”(因为它们都是数据结构, 语义相近), 这就错了。加上 BM25 关键词检索, 就能确保“二叉树”这三个字必须出现。最后通过 RRF 算法进行数学交叉打分, 选出得分最高的前 3 段文本, 这部分文本自带坐标属性, 将直接投喂给大模型。

4.2 基于 DSL 的动态可视化渲染算法

大语言模型通常只能输出线性文本, 无法直接操控 App 的界面元素。为了让算法逻辑(如指针移动、变量交换)以动画形式展现, 我们设计了一套轻量级的 DSL (Domain Specific Language), 要求大模型按照 JSON 格式输出动画步骤, 由前端解析并控制 Canvas 逐帧渲染。

算法伪代码 (Pseudocode) 表示如下:

```
// 算法入口: 前端 Canvas 接收到 DSL 队列并执行渲染
FUNCTION RenderAnimationQueue(dsl_queue):
// 开启异步渲染循环
WHILE dsl_queue IS NOT EMPTY:
// 取出当前指令, 如: { "type": "swap", "target": ["A", "B"], "duration":
500 }
current_action = dsl_queue.Dequeue()
// 解析指令并获取屏幕上的虚拟元素
IF current_action.type == "swap":
element1 = ElementManager.Find(current_action.target[0])
element2 = ElementManager.Find(current_action.target[1])
// 记录起始坐标
start_x1 = element1.x
start_x2 = element2.x
// 启动补间动画引擎 (Tween Animation)
TweenAnimator.animate(
```

```

duration: current_action.duration,
onUpdate: (progress) -> {
// 使用 Lerp（线性插值）计算当前帧的坐标
element1.x = Math.Lerp(start_x1, start_x2, progress)
element2.x = Math.Lerp(start_x2, start_x1, progress)
// 触发 Canvas 重绘
Canvas.markNeedsPaint()
}
)
ELSE IF current_action.type == "highlight":
// 处理高亮、改变颜色等其他指令逻辑...
element = ElementManager.Find(current_action.target[0])
element.color = current_action.color
Canvas.markNeedsPaint()
// 核心同步锁：阻塞队列执行，直到当前动画持续时间结束，避免动画重叠
WaitUntil(current_action.duration)
END WHILE
END FUNCTION

```

常规的 UI 组件框架不适合频繁的像素级重绘。因此我们底层使用了 Flutter 自带的 CustomPaint 画布机制。通过维护一个指令队列（Queue），确保复杂的算法步骤能够被拆解为线性的单一动作。使用 Lerp（线性插值函数）来平滑计算两点之间的过渡状态，保证了 60fps 的流畅视觉体验。队列末尾的 WaitUntil 锁机制非常关键，它确保了“上一步交换动作彻底播完后，才能执行下一步的高亮变色”，以此完美模拟真实程序的单步调试过程。

5、面向对象方法类图的详细设计

本系统的领域模型设计严格遵循关系型数据库的范式。以下为课程管理、文件存储与 AI 交互域的核心类图。

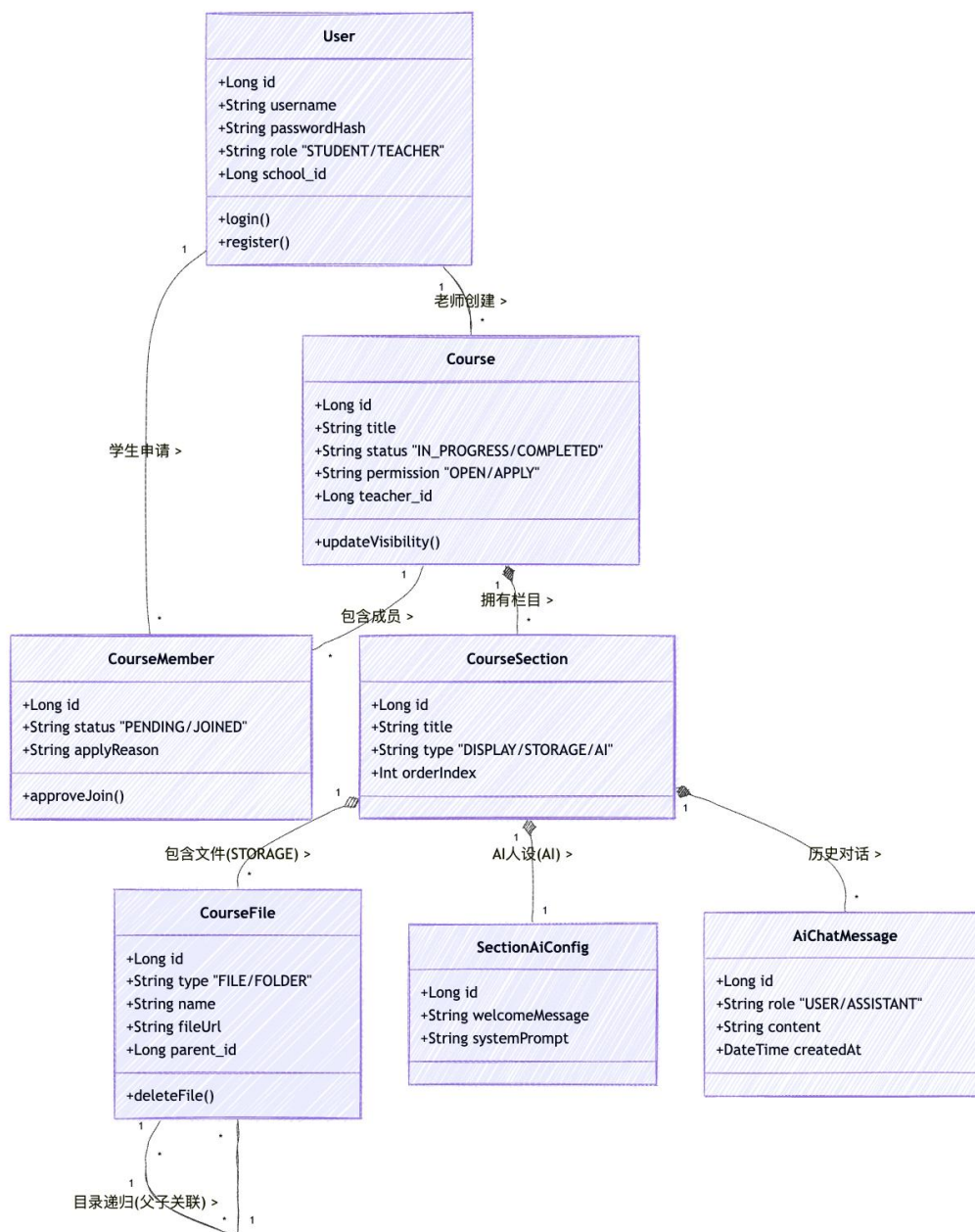


图 5.1 关键类图及其关系图

核心类名	详细说明
User（用户类）	系统的核心参与者。包含基础的凭证信息（username, passwordHash）与角色标识（role）。通过角色枚举（STUDENT/TEACHER）的区分，决定了用户能够访问的接口范围和操作权限。
Course（课程类）	教学活动的主载体。它由教师（即拥有 TEACHER 角色的

	User)	创建，并维护着自身的生命周期状态 (status) 和准入规则 (permission) 。
		这是一个关键的关联类，用于解耦用户与课程的多对多关联。
CourseMember (课 程成员类)		它记录了某个学生加入某门课程的状态 (如 PENDING 待审核, JOINED 已加入)，是系统中接口级越权拦截的核心校验依据。
CourseSection (课 程栏目类)		实现了课程内部模块的可插拔式设计。一门课程可以通过组合模式包含多个栏目 (如资料库、AI 答疑室)，通过 type 字段进行多态区分，前端据此分发渲染不同的 UI 组件。
		文件系统的基石, 采用了经典的组合模式 (Composite Pattern)。
CourseFile (课程文 件类)		通过 parent_id 自关联，一个实体既可以是文件 (FILE)，也可以是文件夹 (FOLDER)，从而在平铺的关系型数据库表中优雅地构建出支持无限层级的树形目录结构。
AiChatMessage (AI 对话记录 类)		专门记录学生与大模型的交互历史。包含消息发送者角色 (USER 提问 或 ASSISTANT 解答) 和具体内容。这些记录不仅用于在移动端渲染聊天气泡流，更是在 RAG 检索时作为“多轮对话上下文”拼装的重要数据源，保证 AI 具有记忆力。

表 6-1 核心类图说明表

6、接口设计

6.1 用户登录接口

请求路径: /api/v1/auth/login/password

请求方式: POST

参数:

username (String, 必填): 用户名

password (String, 必填): 密码 (客户端 RSA 公钥加密)

响应示例:

```
{
  "code": 200,
  "message": "登录成功",
  "data": {
    "token": "eyJhbGciOiJIUzI1...",
    "user_info": { "id": 1, "username": "zhangsan", "role":
"STUDENT" }
  }
}
```

6.2 获取课程文件列表接口

请求路径: /api/v1/courses/{course_id}/sections/{section_id}/files

请求方式: GET

参数:

parent_id (Long, 选填): 目标文件夹 ID, 不传代表请求根目录。

响应内容: 包含 current_folder_id、面包屑路径 path 以及文件列表 list (含文件夹类型 FOLDER 和具体文件 FILE) 。

6.3 发送 AI 提问接口

请求路径: /api/v1/courses/{course_id}/ai/chat

请求方式: POST

参数:

message (String, 必填): 学生的提问内容。

响应示例:

```
{
  "code": 200,
  "message": "回复成功",
  "data": {
    "chat_id": 5001,
  }
}
```

```
    "reply": "动态规划的核心是状态转移方程...\n<coord>[0.1, 0.2, 0.5, 0.3]</coord>",
    "created_at": "2026-03-28T12:01:05Z"
  }
}
```

6.4 申请加入课程接口

请求路径: /api/v1/courses/{course_id}/members/apply

请求方式: POST

参数:

apply_reason (String, 选填): 申请加入课程的理由 (用于教师端审批)。

响应示例:

```
{
  "code": 200,
  "message": "申请已提交，等待教师审批",
  "data": {
    "status": "PENDING"
  }
}
```

6.5 上传课程资料接口

请求路径: /api/v1/courses/{course_id}/sections/{section_id}/files/upload

请求头: Content-Type: multipart/form-data

请求方式: POST

参数:

file (File, 必填): 待上传的课件文件 (如 PDF/PPT)。

parent_id (Long, 选填): 父文件夹 ID, 上传到根目录则不传。

响应示例:

```
{
  "code": 200,
  "message": "文件上传并解析成功",
}
```

```
"data": {
  "file_id": 1024,
  "file_name": "第六章_动态规划.pdf",
  "file_url": "https://oss-storage.../file.pdf"
}
```

6.6 配置课程 AI 人设接口

请求路径: /api/v1/courses/{course_id}/sections/{section_id}/ai-config

请求方式: PUT

参数:

welcome_message (String, 必填): 答疑室欢迎界面的开场白。

system_prompt (String, 必填): 在后端组装时传递给大模型的基础人设 Prompt。

响应示例:

```
{
  "code": 200,
  "message": "AI 人设配置更新成功",
  "data": null
}
```

7、数据库物理设计

我们系统采用 PostgreSQL 进行数据持久化，表结构经过严格的范式设计，并合理使用了外键和索引以保障数据一致性。为了支撑多维度业务与 AI 功能，核心数据表设计如下：

7.1 学校与机构表 (schools)

记录平台入驻的高校或机构信息，用于主页的课程分类和租户数据隔离。

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	学校唯一 ID
name	VARCHAR(100)	UNIQUE, NOT NULL	学校或机构全称

domain	VARCHAR(100)	NULL	学校域名或专属后缀
--------	--------------	------	-----------

7.2 用户表 (users)

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	用户唯一主键
username	VARCHAR(32)	UNIQUE, NOT NULL	登录账号名
password_hash	VARCHAR(100)	NOT NULL	哈希加密后的密码
role	VARCHAR(16)	CHECK	角色 (STUDENT / TEACHER)
school_id	BIGINT	FOREIGN KEY	关联 schools 表

7.3 课程表 (courses)

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	课程唯一 ID
teacher_id	BIGINT	FOREIGN KEY	关联 users 表 (教师)
title	VARCHAR(255)	NOT NULL	课程名称
status	VARCHAR(20)	CHECK	预发布/进行中/完结/隐藏
permission	VARCHAR(20)	CHECK	准入机制 (OPEN / APPLY)

7.4 课程成员表 (course_members)

解耦多对多关系的中间表，用于记录学生的报名审批状态。

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	记录 ID
course_id	BIGINT	FOREIGN KEY	关联 courses 表

user_id	BIGINT	FOREIGN KEY	关联 users 表 (学生)
status	VARCHAR(20)	CHECK	PENDING (待审) / JOINED (已入)
apply_reason	VARCHAR(500)	NULL	申请加入理由

7.5 课程栏目表 (course_sections)

支持动态多态配置，一门课程可包含多个不同功能的栏目视图。

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	栏目 ID
course_id	BIGINT	FOREIGN KEY	归属课程 ID
title	VARCHAR(100)	NOT NULL	栏目标题名称
type	VARCHAR(20)	CHECK	DISPLAY(公告) / STORAGE(文件) / AI
order_index	INT	NOT NULL	渲染排序权重

7.6 课程文件表 (course_files)

此表通过 parent_id 字段自关联，实现了计算机系统中经典的树形目录结构。

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	文件/文件夹 ID
section_id	BIGINT	FOREIGN KEY	归属的课程栏目 ID
parent_id	BIGINT	FOREIGN KEY	父文件夹 ID (空表示根目录)
type	VARCHAR(20)	CHECK	FILE (文件) / FOLDER (文件夹)
name	VARCHAR(255)	NOT NULL	显示名称
file_url	VARCHAR(255)	NULL	对象存储下载链接

7.7 AI 对话记录表 (ai_chat_messages)

持久化保存问答历史，作为长程记忆（Long-term memory）组装历史对话 Context。

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	消息 ID
section_id	BIGINT	FOREIGN KEY	归属的 AI 栏目 ID
user_id	BIGINT	FOREIGN KEY	提问学生 ID
role	VARCHAR(20)	CHECK	USER (用户) / ASSISTANT (大模型)
content	TEXT	NOT NULL	消息内容或 DSL 结构体
created_at	TIMESTAMP	NOT NULL	发送时间

7.8 知识切片向量表 (knowledge_chunks)

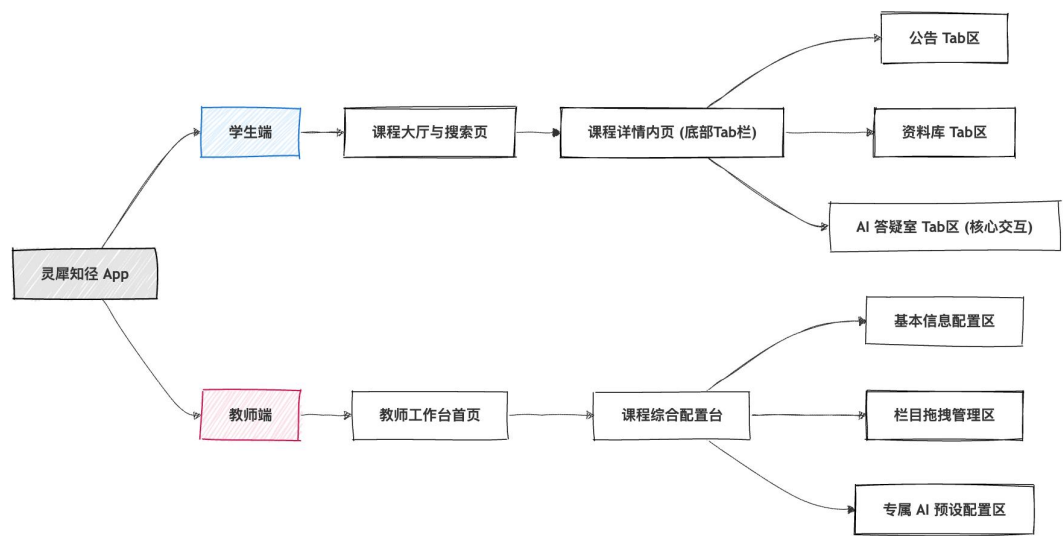
RAG 算法的核心底层存储，利用 pgvector 插件进行高维向量空间计算。

字段名	类型	约束/索引	描述
id	BIGSERIAL	PRIMARY KEY	切片 ID
file_id	BIGINT	FOREIGN KEY	源文件 ID (关联 course_files)
page_num	INT	NOT NULL	所属课件的页码
text_content	TEXT	FTS INDEX	OCR 提取出的原文本 (支持全文检索)
bbox_coords	JSONB	NOT NULL	原文本在课件上的绝对位置坐标框
text_embedding	VECTOR(1024)	HNSW INDEX	高维语义向量 (支持高效余弦搜索)

8、UI（界面）设计

为了让老师和学生都能快速上手，我们在 Flutter 跨平台框架下对移动端 App 界面进行了克制且清晰的设计。通过解耦双角色的操作链路，我们将整个

UI 导航结构设计如下：



8.1 学生端主干 UI 说明

针对知识消费者 (学生)，界面的核心诉求是发现课程和沉浸式学习与答疑。

界面视图名称	UI 布局与交互功能说明
课程大厅与搜索页	入口视图：进入 App 的首屏。顶部悬浮搜索框和“本校课程”快捷筛选器。主体采用支持下拉刷新的瀑布流列表设计。每个卡片直观展示课程的封面图、标题、任课老师姓名以及准入状态角标（例如绿色标识的“可直接加入”或黄色标识的“需申请”）。
课程详情 - 公告 Tab	静态阅读区：学生进入具体的课程后，底部为固定导航栏。首个标签页用于展示老师发布的教学大纲或考试说明。底层接入 Markdown 渲染引擎，支持富文本、加粗、引用等格式的优雅解析。
课程详情 - 资料库 Tab	网盘资源区：采用仿原生系统文件管理器的 UI 设计，展示规整的文件夹与文件图标。交互上，点击文件夹通过推入式动画 (Push) 平滑切换至下一级目录；点击具体文件 (如 PDF、MP4) 则直接唤起系统底层级组件全屏预览或触发下载至本地沙盒。
课程详情 - AI 答疑室 Tab	跨模态核心区：界面采用动态的上下分屏或抽屉弹出层布局。上方是类似微信的聊天消息气泡流，承载学生提问与 AI 文字回复；下方是动态画布与文档视图区。当 AI 回复中带有课件坐标

标识时，下方视图引擎会自动翻到对应的 PDF 页码，并在屏幕上精准计算物理像素位置，动态闪烁黄色的高亮遮罩，实现完美的图文对照。

8.2 教师端主干 UI 说明

针对知识创作者 (教师)，界面的核心诉求是资源管理效率和配置的灵活性。

界面视图名称	UI 布局与交互功能说明
教师专属工作台	管理看板：教师角色的专属首屏，采用卡片式列表纵向展示该老师创建的所有历史与在授课程，右上角提供醒目的“创建新课程”悬浮按钮 (FAB)。此外，若有学生正在申请加入设置了门槛的课程，主页将通过醒目的红色气泡角标提示老师进行待办审批。
课程综合配置台	控制中枢：整合了课程生命周期内的所有核心设定。界面横向划分为三个子版块：基本信息面板（修改封面/标题/准入规则）、学生管理面板（踢出或通过审核）、以及栏目管理面板。其中栏目管理支持通过拖拽 (Drag and Drop) 调整栏目显示顺序，或点击“+”号新增多态模块。
AI 栏目专属配置器	大模型人设后台：点击进入特定 AI 答疑栏目设置时，界面呈现两个高度定制化的核心多行文本输入框。其一是“欢迎语配置”，定义学生首次打开界面的寒暄文案；其二为极度关键的“系统提示词 (System Prompt) 设定框”。系统在此处通过强引导提示老师使用自然语言输入规则（例如“无论如何不要给出代码答案，请用反问引导他”），这一段预设的“紧箍咒”对学生完全不可见，只在底层组装时悄悄传递给大模型。